CtrlEat — Milestone 3 Architecture & Reflection

Layered Service Development with AI Agents | Evidence snapshot: November 10, 2026

1. Architecture and frontend integration

CtrlEat began as an ASP.NET Core REST application with controllers, services, models and JSON repositories. Milestone 3 separates capabilities into independent service projects: **Domain** (framework-independent business rules and entities), **Data** (EF Core, SQLite, repositories and migrations), **API** (HTTP endpoints, public DTO contracts, orchestration and bearer-token security), and **Client** (web interfaces using HTTP/REST, without direct database or domain-assembly access). The dependency direction is Domain → Data → API as consumers, with the browser client accessing public API endpoints.

The **User** capability is implemented in User.Domain/Data/Api with a React/Vite User.Client. Its OIDC code flow with PKCE authenticates the user, and its API stores a validated positive calorie goal per identity. The **Daily Meal Plan** capability is implemented in DailyMealPlan.Domain/Data/Api/Client with ASP.NET Core Razor Pages, a SQLite database and external Recipe/Ingredient HTTP lookups. Invalid references are rejected before writes. This implementation is on open PR #16, not yet integrated.

Storage Location and Grocery List have separate Domain/Data/Api implementations, data migrations, tests and frontend files merged in PR #18. Existing legacy Recipe/Ingredient endpoints support local lookups; the two separate layered Milestone 3 services and clients for these capabilities have not been verified. This remains a substantive milestone gap, not merely a documentation issue.

2. Work allocation and contract boundaries

Owner	Services and tracked evidence	Current integration state
Félix	User: issue #12 / PR #13; Daily Meal Plan: issue #14 / PR #16	User merged; Meal Plan open and conflicting
Zachary	Storage Location and Grocery List: merged PR #18	Layered code integrated; owner verification still needed
Sarah	Ingredient and Recipe: legacy PR #1; ingredient enhancement issue #19	Standalone layered services not verified

The team should confirm shared request/response contracts, endpoint scopes, and downstream service identities before production integration. Work items must show 3–6 granular tasks, ownership and acceptance criteria for all six services.

3. Specialized AI agents and manual corrections

Development follows the **Research** → **Plan** → **Implement** → **Validate (R-PIV)** loop. For issue #12, the repository Copilot Planner generated an implementation plan; Félix approved separate service-layer projects and a minimal calorie-goal DTO. Domain, Data, API and Frontend agents implemented the feature, while developer-led validation corrected accidental nested-project compilation and the identity-provider subject-claim configuration.

For issue #14, the approved plan guided sequential layer implementation. A human selected Razor Pages rather than the initial React task proposal, and follow-up implementation addressed the User client's misleading failed-profile state and earlier build warnings. Narrowly scoped agents accelerated layer-by-layer implementation and focused checks, but did not replace judgment about contracts, provider settings, failing lookups, usability and deployment risks. The other team members must document their own AI workflows and corrections; they have not been inferred from their committed code.

4. Human validation gates: strengths and limits

Gate 1 — specification and plan: Issue #12 has a saved Planner report and explicit approval; issue #14 also links approved planning decisions. This helps keep business acceptance criteria visible before coding. Comparable plan approval evidence remains necessary for every other work item.

Gate 2 — verification: For PR #16 revision af17fbd relative to a54f2fb, the independent Validator recorded a warning-free .NET build, 21/21 tests passing, clean diff checks and no actionable findings. A prior browser demonstration tested OIDC sign-in, create/reload, reference rejection with retained input, date editing and deletion. User isolation and dependency-failure behavior were tested in the API suite. These results belong to the reviewed revision and **cannot certify the newer combined branch**.

Gate 3 — peer signoff: Zachary approved PR #16. However, Félix's personal live verification and line-by-line diff review remain outstanding. PR #16 currently conflicts with recent changes, including PR #18. Resolve conflicts carefully, rerun the entire integrated solution and frontend tests, and record a real human signoff on the resolved code before merging.

Overall, independent validation exposed assumptions and supplied repeatable regression evidence. The principal remaining risks are divergent integration branches, unverified Ingredient/Recipe layering, incomplete team-wide work tracking and service-specific security evidence.

Completion and submission notes

The milestone requires at least two independently runnable layered services per member; a published work tracking board; custom AI agents/skills; OAuth2/OIDC bearer policies and tests; merged work-item PRs with human gates; and the required Azure DevOps milestone-level PR. The milestone branch must be retained after merge, and the Azure DevOps PR link submitted on Lea. This report documents verified progress but does not assert that these remaining steps are complete.

Supporting repository documents: Docs/Milestone-3-tracking.md, Docs/Milestone-3-authentication-and-validation.md and Docs/Milestone-3-team-evidence.md. Sources: GitHub PRs #13, #16, #18 and issues #12, #14, #19.